

# Mapping To and From FHIR

Wearables Project | TU Berlin and Charité Berlin



# Requirements

- We need to be able to export (at least) in FHIR
- We don't want to generate FHIR from incoming JSON but from Influx
- We need to consider different incoming data and formats / schemas
- We want to define a declarative approach for (de-)serialization from and to JSON in a way that does not allow configuration drift



## Current Approach

- We declaratively describe with a YAML (k8s ConfigMap):
  - How the incoming data should be mapped to FHIR and validated
  - What data is written to Influx
- We want to store Metadata (Patients, Watches, Devices etc.) in Postgres

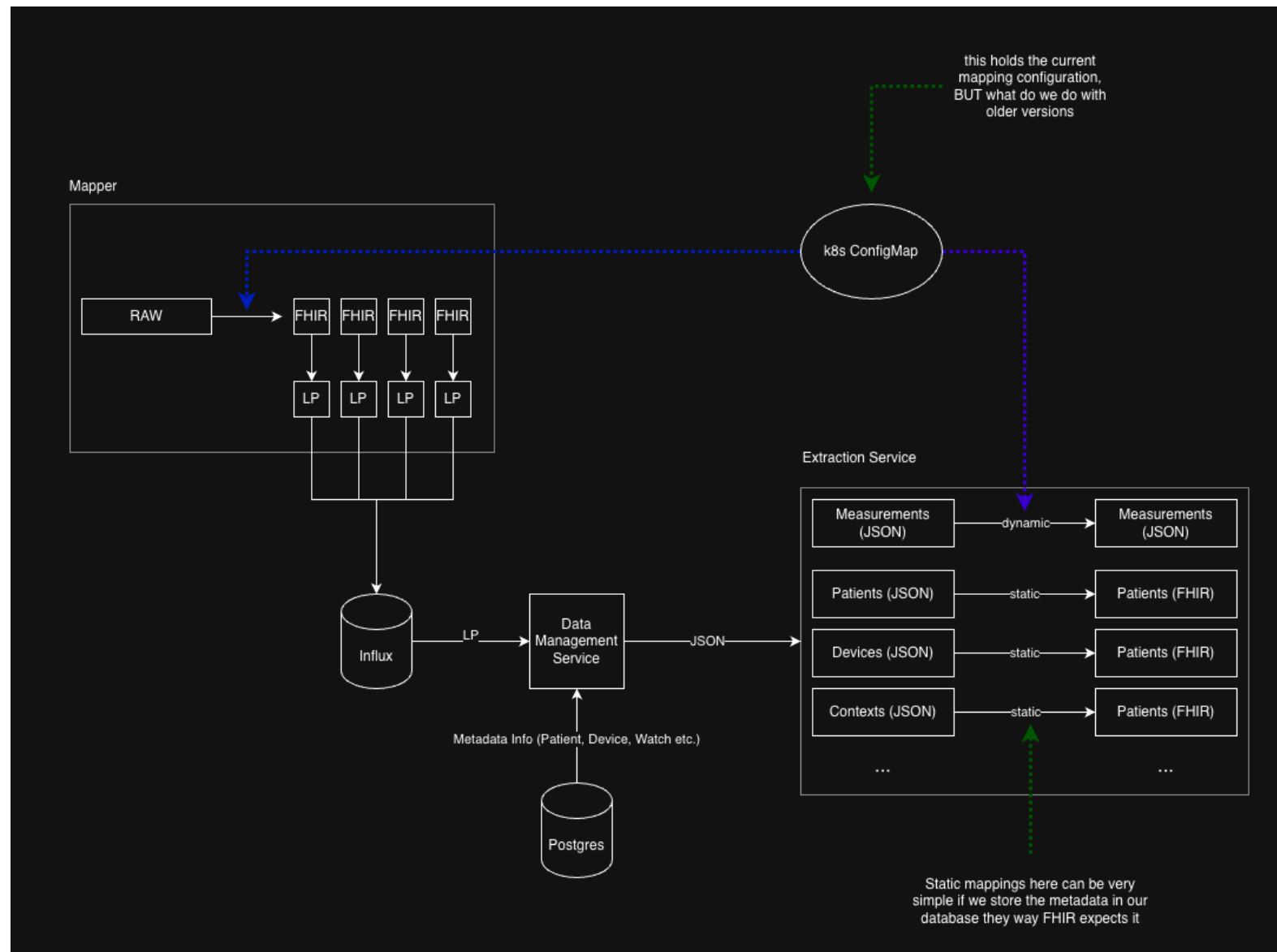


# Challenges

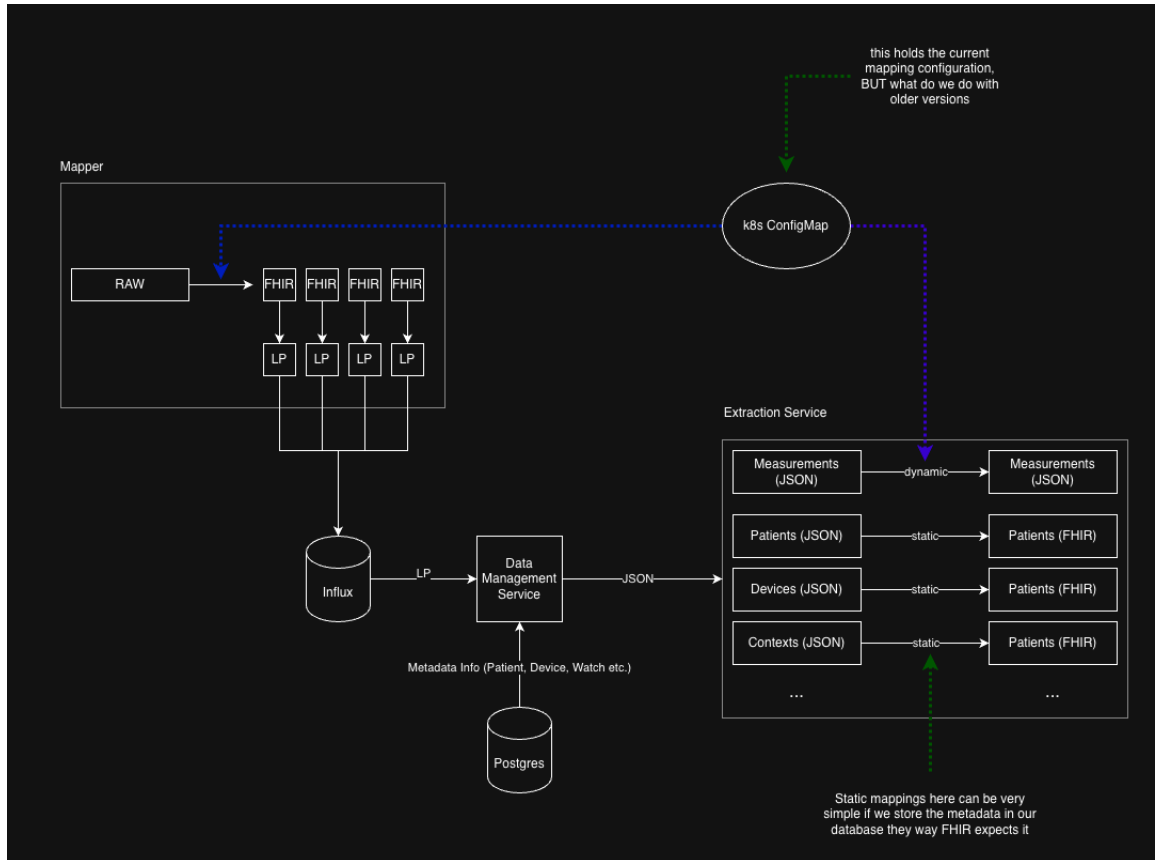
- To leverage the flexibility in our approach with the new setup we need to think about how we will handle serializing Line Protocol back to FHIR
  - What happens if the transformation rules change?
  - Where do we want to store old rules?
- What FHIR resources best fit our metadata entities?
- What metadata resources do we want to map to FHIR?
  - What about context, case etc.



# Proposal



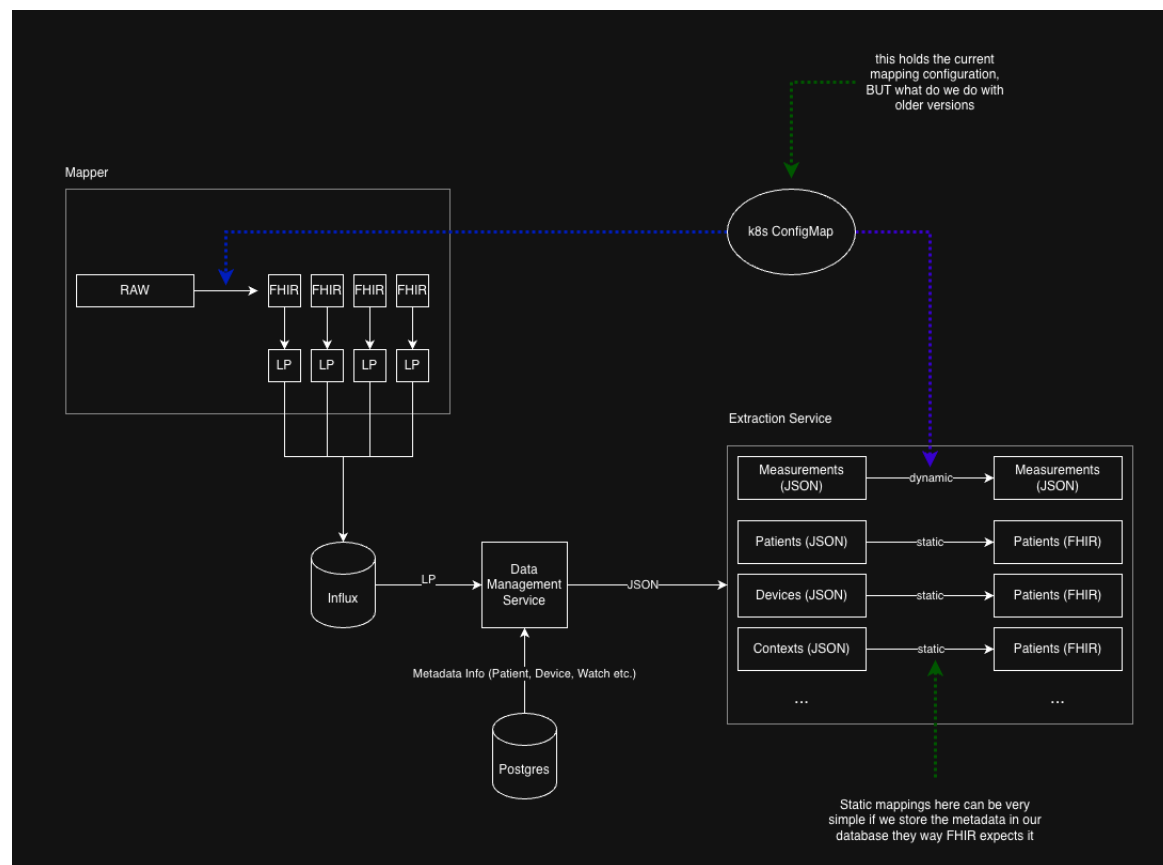
# Proposal: Metadata



- Metadata entities are stored in Postgres with static schema
  - Does not allow for easy changes
  - Makes our job easier as mappings can be “static”
- We can store data in Postgres the way it should be in FHIR



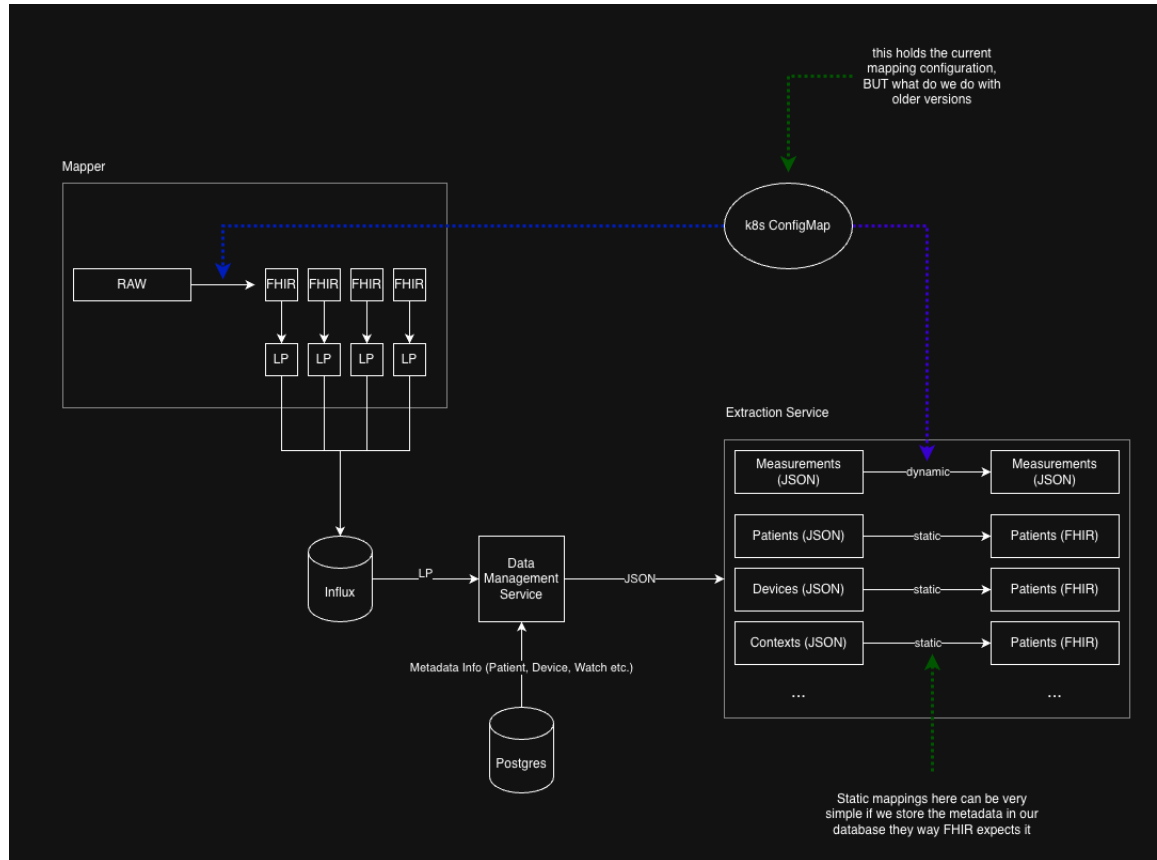
# Proposal: Bidirectional mapping



- Measurements are more complicated because of flexibility / dynamic approach
- Idea: Unified YAML that allows mapping in both directions
  - JSON → FHIR → LP
  - LP → FHIR
- This gives the responsibility of reconstructability to the person that defines the YAML → Validation?



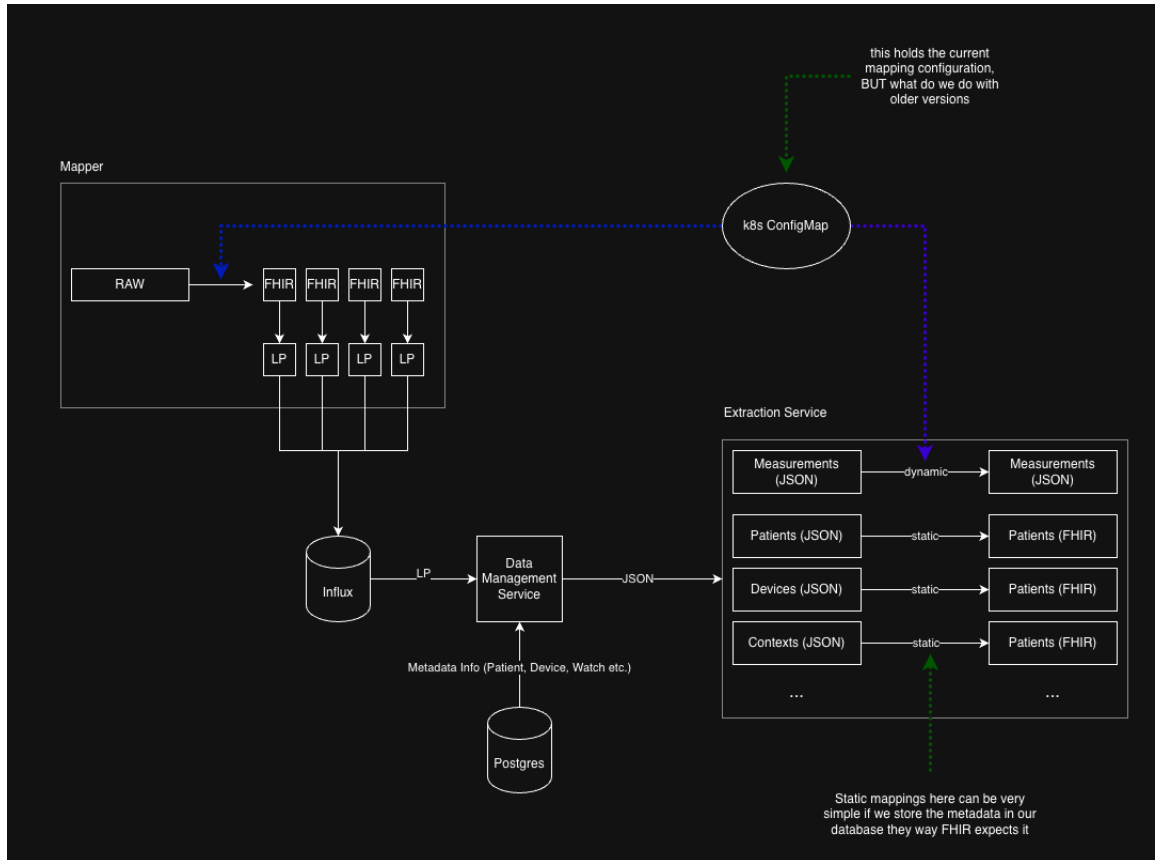
# Proposal: Bidirectional mapping



- Normally implementation detail but not here since shared responsibility
- Generally: the more freedom we give, the more complicated it gets



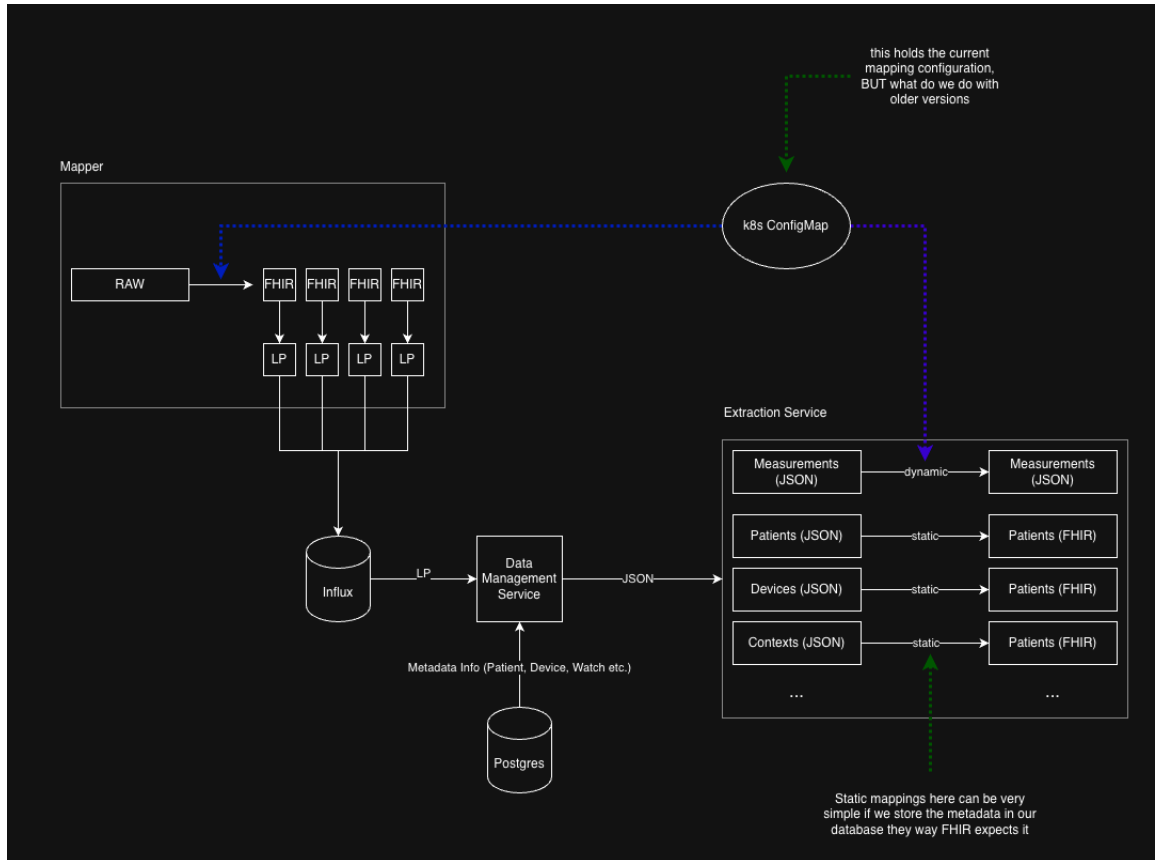
# Proposal: Versioning



- Incoming data may change → YAML may need to change (in ConfigMap)
  - Old measurements can no longer be serialized since YAML is no longer valid for those measurements
- We need to store old **versions** of the YAML → add (semantic) version field to measurement that describes how it can be deserialized



# Proposal: Versioning



- Where do we store older versions?
  - Postgres → unsuitable (?)
  - (multiple) ConfigMaps → impractical (?)
  - In-mem of Extraction Service (with periodic / triggered / gossiped config changes)
- Who queries versions?
  - Data Management Service → then sends them to the Extraction Service when needed
  - Extraction service does Lookup as needed



## Additionally...

- If we want to export in other formats:
  - What formats? (CSV, LP, normal JSON etc.)



## Extra: Mapping with FHIR library

```
import json
from fhir.resources.patient import Patient
from fhir.resources.humanname import HumanName
from fhir.resources.contactpoint import ContactPoint

patient = Patient.model_validate_json("""{
  "resourceType": "Patient",
  "name": [
    {
      "family": "Doe",
      "given": [
        "Jay"
      ]
    }
  ],
  "telecom": [
    {
      "system": "email",
      "value": "example@abc.com"
    }
  ],
  "gender": "male"
}
""")

print(patient.model_dump_json(indent=2))
```

- Does not work since there is no way to know which fields need to be set
- Maybe possible with reflections but that's dangerous
- Same problem I originally ran into in the Streams instance
- Instead: Manually build a JSON (with support of a library) and validate that its semantically and syntactically correct

